

# Database Management Systems

## Lecture Study Notes — Transaction Schedules & Execution Models

### 1. WHAT IS A SCHEDULE?

A **schedule** is the chronological execution sequence of operations from multiple transactions.

#### Core Concept

A schedule defines the exact order and timing in which statements from different transactions are processed by the DBMS engine.

**Example Scenario:** Consider three distinct transactions: T1, T2, and T3.

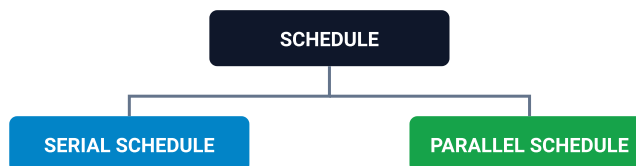
If the system executes all operations of T1 first, followed by all of T2, and finally T3, the resulting ordered sequence:

T1 → T2 → T3

is formally called a **transaction schedule**.

### 2. TAXONOMY OF SCHEDULES

Transaction schedules are categorized into two fundamental execution paradigms based on how operations are interleaved:



### 3. SERIAL SCHEDULE

In a **serial schedule**, one transaction completes its entire execution cycle before any subsequent transaction is allowed to begin. No operations are interleaved.

#### Execution Sequence

NON-INTERLEAVED

T1 (Starts & Finishes)

↓

T2 (Starts & Finishes)

↓

T3 (Starts & Finishes)

**Strict Rule:** Once a transaction starts, no other transaction can interfere until it commits or aborts.

#### Key Characteristics

- **Interleaving:** ❌ Strictly prohibited.
- **Isolation:** Maximum isolation between transactions.
- **Consistency:** High guaranteed correctness.
- **Resource Utilization:** Low (CPU idle during I/O).

### Real-Life Analogy: Single ATM Counter (Serial Model)

Imagine a single physical ATM with three customers waiting in line (Person A, B, and C). Person A must finish their entire transaction before Person B can insert their card. Person C waits behind B. This single-queue model ensures zero interference but results in high cumulative waiting time.

## 5. PARALLEL (CONCURRENT) SCHEDULE

In a **parallel (or concurrent) schedule**, operations from multiple transactions are interleaved over time. The CPU context-switches between active transactions to allow all of them to make simultaneous progress.

### Interleaved Execution Flow

CONCURRENT

T1 [Op 1] → T2 [Op 1] → T3 [Op 1] → T1 [Op 2] → T2 [Op 2]

The operating system / DBMS CPU scheduler rapidly rotates execution slots across transactions, avoiding prolonged queue delays.

### Real-Life Analogy: Online Banking & IRCTC (Parallel Model)

In online platforms like IRCTC or net banking, thousands of users initiate ticket bookings or fund transfers simultaneously. The database processes read/write operations concurrently rather than queuing millions of users sequentially.

## 6. UNDERSTANDING THROUGHPUT

System efficiency in transaction processing is evaluated primarily through **throughput**.

### THROUGHPUT METRIC

$$\text{Throughput} = \frac{\text{Total Transactions Executed}}{\text{Unit of Time}}$$

Example: If 100 transactions commit successfully in 1 second, Throughput = **100 TPS** (Transactions Per Second).

Parallel Execution ⇒ Reduced Wait Time ⇒ Higher TPS ⇒ Superior Performance

## 7. SERIAL VS. PARALLEL SCHEDULES COMPARISON

### Exam Cheat Sheet & Memory Tricks:

- **SERIAL = QUEUE:** T1 → T2 → T3 (Simple, safe, but slow).
- **PARALLEL = SWITCH:** T1 → T2 → T1 → T3 (Fast, efficient, but requires conflict handling).
- **Trade-Off:** Parallel execution improves throughput but can introduce concurrency anomalies (e.g., Dirty Reads, Lost Updates) which are resolved using serializability protocols.

| Feature        | Serial Schedule           | Parallel Schedule                 |
|----------------|---------------------------|-----------------------------------|
| Execution Mode | One transaction at a time | Multiple transactions interleaved |